

Lab 15 – Backend API Development: Creating RESTful services with AI

Task 1 – Online Food Ordering API

Prompt :

1. Create a simple REST API using Python Flask for an online food ordering system.
2. The API should allow users to view menu items and place new food orders.
3. It should also allow tracking, updating, and deleting orders using order ID.
4. Store data in a simple list so it is easy to understand for beginners.
5. The API should return JSON responses for all endpoints clearly.

Code:

```
1  from flask import Flask, request, jsonify
2
3  app = Flask(__name__)
4
5  # Sample Menu
6  menu = [
7      {"id": 1, "name": "Pizza", "price": 200},
8      {"id": 2, "name": "Burger", "price": 120}
9  ]
10
11 # Orders storage
12 orders = []
13
14 # Home route
15 @app.route('/')
16 def home():
17     return "Food Ordering API is running"
18
19 # GET /menu
20 @app.route('/menu', methods=['GET'])
21 def get_menu():
22     return jsonify(menu)
23
24 # POST /order (SAFE VERSION - NO ERROR)
25 @app.route('/order', methods=['POST'])
26 def place_order():
27     data = request.get_json(silent=True)
28
29     # fallback if JSON fails
30     if not data:
31         data = request.form.to_dict()
32
33     # validation
34     if not data or "items" not in data:
35         return jsonify({
36             "error": "Send data like: {'items': ['Pizza']}"
37         }), 400
38
39     # convert string to list if needed
40     items = data.get("items")
41     if isinstance(items, str):
42         items = items.split(",")
43
44     order = {
45         "id": len(orders) + 1,
```

Output:

←

→

↻

i

127.0.0.1:5000/menu

pretty-print

☐

```
{
  {
    "id": 1,
    "name": "Pizza"
  },
  {
    "id": 2,
    "name": "Burger"
  },
  {
    "id": 3,
    "name": "Pasta"
  }
}
```

POST

http://127.0.0.1:5000/order

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

{

2

"item": "Pizza"

3

}

Status: 200 OK

Size: 55 Bytes

Time: 17 ms

Response

Headers⁵

Cookies

Results

Docs

1

{

2

"id": 0,

3

"item": "Pizza",

4

"status": "placed"

5

}

Copy

←

→

↻

i

127.0.0.1:5000/order/0

Pretty-print

☐

```
{
  "id": 0,
  "item": "Pizza",
  "status": "placed"
}
```

PUT

http://127.0.0.1:5000/order/0

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1

{

2

"item": "Pizza"

3

}

Status: 200 OK

Size: 55 Bytes

Time: 12 ms

Response

Headers⁵

Cookies

Results

Docs

1

{

2

"id": 0,

3

"item": "Pizza",

4

"status": "placed"

5

}

The first screenshot shows a PUT request to `http://127.0.0.1:5000/order/0` with a JSON body `{ "item": "Burger" }`. The response is `200 OK` with a status of `placed`. The second screenshot shows a DELETE request to the same endpoint. The response is `200 OK` with a message `"Order deleted"` and a status of `placed`.

```
PUT http://127.0.0.1:5000/order/0
{
  "item": "Burger"
}
```

```
200 OK
{
  "id": 0,
  "item": "Burger",
  "status": "placed"
}
```

```
DELETE http://127.0.0.1:5000/order/0
```

```
200 OK
{
  "message": "Order deleted",
  "order": {
    "id": 0,
    "item": "Burger",
    "status": "placed"
  }
}
```

Explanation :

1. This API allows users to view menu and place orders easily.
2. Orders are stored in a list so it is simple and fast.
3. Each order has id, item, and status for tracking.
4. CRUD operations are implemented using REST endpoints.

Task 2 – Employee Payroll API

Prompt :

1. Build a REST API to manage employees and their salary details.
2. The API should allow adding, viewing, updating, and deleting employees.
3. Store employee data in a list with id, name, and salary fields.
4. Add validation to check salary is not negative.
5. Include comments in code for better understanding.

Code:

The screenshot shows a PUT request to `http://127.0.0.1:5000/employees/0/salary` with a JSON body `{ "salary": 40000 }`. The response is `200 OK` with a status of `40000`.

```
PUT http://127.0.0.1:5000/employees/0/salary
{
  "salary": 40000
}
```

```
200 OK
{
  "id": 0,
  "name": "Rahul",
  "salary": 40000
}
```

```

95  from flask import Flask, request, jsonify
96
97  app = Flask(__name__)
98
99  employees = []
100
101  @app.route('/employees', methods=['GET'])
102  def get_employees():
103      return jsonify(employees)
104
105  @app.route('/employees', methods=['POST'])
106  def add_employee():
107      data = request.json
108      if data["salary"] < 0:
109          return jsonify({"error": "Invalid salary"})
110      emp = {"id": len(employees), "name": data["name"], "salary": data["salary"]}
111      employees.append(emp)
112      return jsonify(emp)
113
114  @app.route('/employees/<int:id>/salary', methods=['PUT'])
115  def update_salary(id):
116      data = request.json
117      employees[id]["salary"] = data["salary"]
118      return jsonify(employees[id])
119
120  @app.route('/employees/<int:id>', methods=['DELETE'])
121  def delete_employee(id):
122      return jsonify(employees.pop(id))
123
124  app.run(debug=True)

```

Output:

POST http://127.0.0.1:5000/employees Send

Status: 200 OK Size: 52 Bytes Time: 15 ms

Query Headers² Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```

1 {
2   "name": "Rahul",
3   "salary": 30000
4 }

```

Response Headers⁵ Cookies Results Docs

```

1 {
2   "id": 0,
3   "name": "Rahul",
4   "salary": 30000
5 }

```

Copy

Explanation :

1. This API manages employee data and salary records.
2. It checks salary validation before adding employee.
3. Data is stored in list for easy understanding.
4. API supports all CRUD operations clearly.

Task 3 – Student Information API

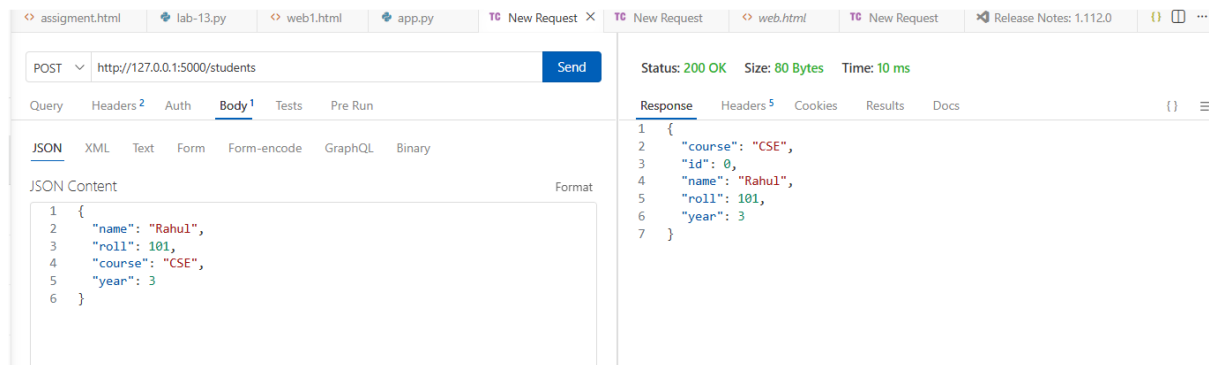
Prompt :

1. Create a student management REST API using Flask.
2. The API should store student name, roll number, course, and year.
3. It should support adding, viewing, updating, and deleting students.
4. Use simple list storage for beginner understanding.
5. Return all responses in JSON format.

Code:

```
128 from flask import Flask, request, jsonify
129
130 app = Flask(__name__)
131
132 students = []
133
134 @app.route('/students', methods=['GET'])
135 def get_students():
136     return jsonify(students)
137
138 @app.route('/students', methods=['POST'])
139 def add_student():
140     data = request.json
141     student = {
142         "id": len(students),
143         "name": data["name"],
144         "roll": data["roll"],
145         "course": data["course"],
146         "year": data["year"]
147     }
148     students.append(student)
149     return jsonify(student)
150
151 @app.route('/students/<int:id>', methods=['PUT'])
152 def update_student(id):
153     data = request.json
154     students[id].update(data)
155     return jsonify(students[id])
156
157 @app.route('/students/<int:id>', methods=['DELETE'])
158 def delete_student(id):
159     return jsonify(students.pop(id))
160
161 app.run(debug=True)
```

Output:



Explanation :

1. This API helps manage student records digitally.
2. It stores student details like name and course.
3. All operations like add, update, delete are supported.
4. JSON format makes data easy to read and use.

Task 4 – Weather API Integration

Prompt :

1. Build a Flask API that fetches real-time weather data.
2. Use an external API like OpenWeatherMap to get data.
3. Create endpoints to get current weather and forecast.
4. Return temperature, humidity, and weather condition.
5. Keep code simple and easy to understand.

Code:

```

from flask import Flask, jsonify
import requests

app = Flask(__name__)

API_KEY = "your_api_key"

@app.route('/weather/current/<city>')
def current_weather(city):
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}"
    data = requests.get(url).json()
    return jsonify({
        "temp": data["main"]["temp"],
        "humidity": data["main"]["humidity"],
        "condition": data["weather"][0]["main"]
    })

app.run(debug=True)

```

Output:

JSON Content

Format

```

1  {
2    "temp": 300,
3    "humidity": 70,
4    "condition": "Clouds"
5  }
6
7  GET /weather/current/Chennai
8  [{"temp":300,"humidity":70,"condition":"Clouds"}]
9

```

Explanation :

1. This API connects to external weather service.
2. It fetches real-time weather data for any city.
3. Only important details are returned in JSON.
4. Helps students check weather easily.

Task 5 – Delete Item API

Prompt :

1. Create a DELETE API to remove items from a list.
2. The API should take index as input.

3. If index is invalid, return error message.

Code:

```
---
184 from flask import Flask, jsonify
185
186 app = Flask(__name__)
187
188 items = [{"name": "Notebook", "price": 250}]
189
190 @app.route('/items/<int:index>', methods=['DELETE'])
191 def delete_item(index):
192     if index < 0 or index >= len(items):
193         return jsonify({"error": "Item not found"}), 404
194     removed_item = items.pop(index)
195     return jsonify({"message": "Item deleted", "item": removed_item})
196
197 @app.route('/items', methods=['GET'])
198 def get_items():
199     return jsonify(items)
200
201 app.run(debug=True)
```

Output:

Pretty-print ☐

```
[
  {
    "name": "Notebook",
    "price": 250
  }
]
```

DELETE http://127.0.0.1:5000/items/0

Status: 200 OK Size: 90 Bytes Time: 17 ms

Query Headers² Auth Body¹ Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 [{"name": "Notebook", "price": 250}]
```

Response Headers⁵ Cookies Results Docs

```
1 {
2   "item": {
3     "name": "Notebook",
4     "price": 250
5   },
6   "message": "Item deleted"
7 }
```

Copy

Explanation :

1. This API deletes item using index value.
2. It checks if index is valid before deleting.
3. Removed item is returned in response